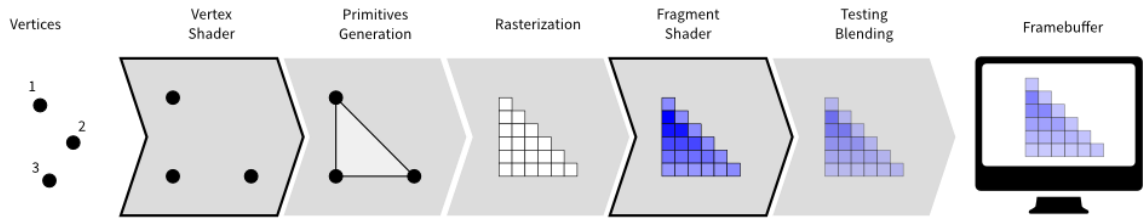
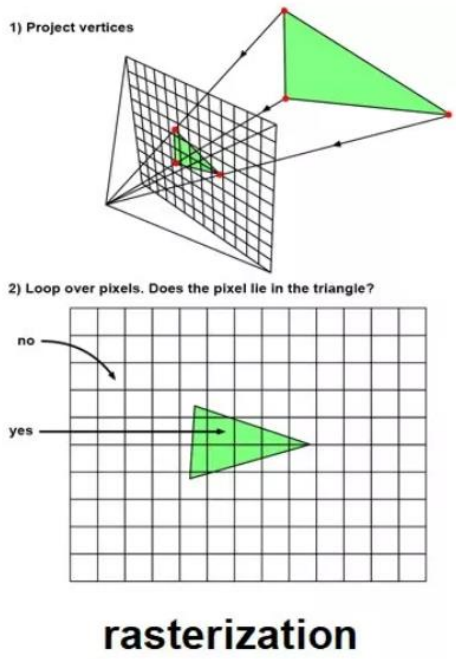
Quelques précisions sur le pipeline d'OpenGL

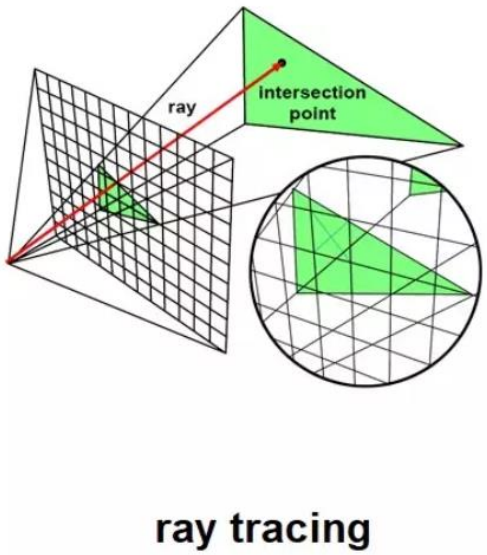
Malek.Bengougam@gmail.com

Des sommets aux pixels

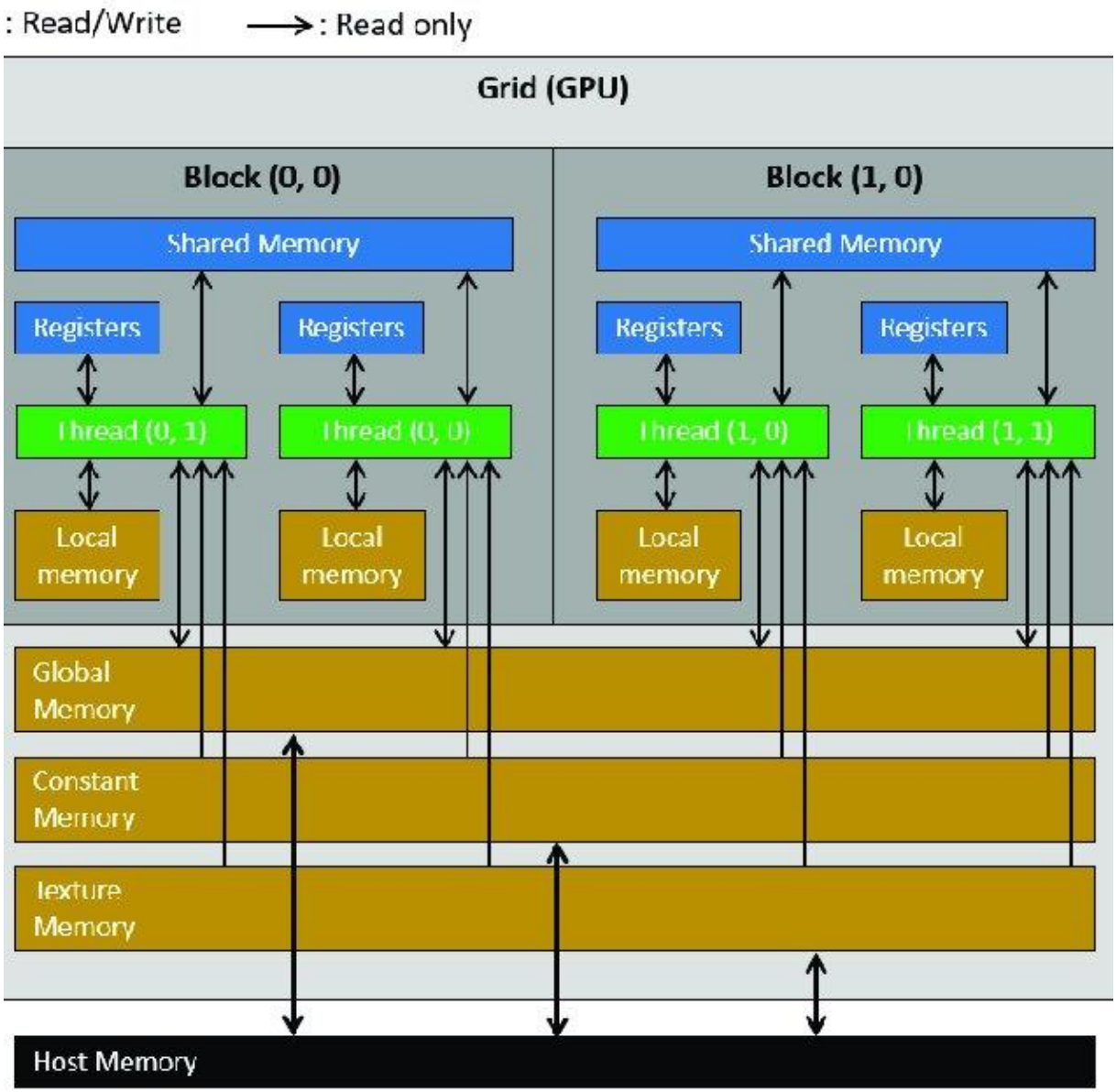


La *rasterization* est une méthode de remplissage qui génère chaque fragment après projection. Contrairement à des méthodes comme le lancer de rayon (*ray tracing*) l'écriture du pixel est la dernière étape du pipeline, alors qu'en lancer de rayon on par du pixel et on cherche (par calculs d'intersections et rebonds) qu'elles sont les « couleurs » qui viennent influencer ce pixel.





Une vue interne du GPU

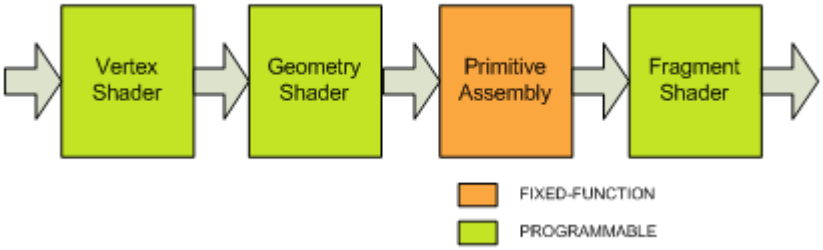


On note que les différentes mémoires du GPU (Global Memory, etc…) peuvent communiquer en lecture et écriture avec la mémoire système (ici appelée « Host Memory »

La « Constant Memory » correspond à la mémoire dédiée aux constantes, qu'OpenGL appelle « **uniform** ». Cette zone mémoire est assez faible (grand maximum 64 kio) et son contenu est volatile : il peut changer d'un shader (ou appel de rendu) à un autre.

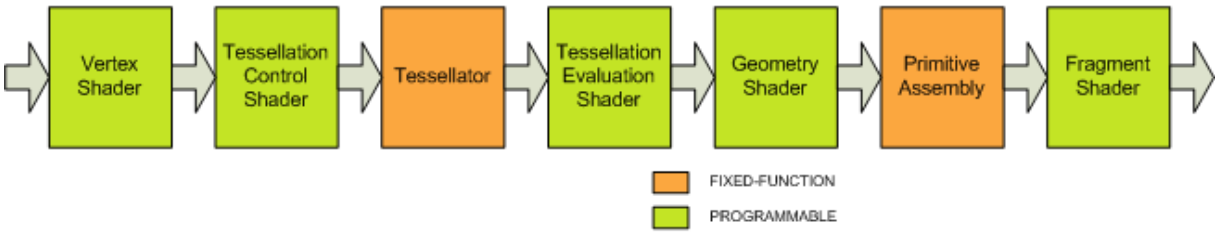
La « Global Memory » et la « Texture Memory » sont, elles, persistantes et indépendantes du shader program courant, car cette zone est gérée par allocation mémoire – via les appels à glBufferData() ou glTexImage2D() par exemple.

Nouveauté du pipeline en OpenGL 3.



Un nouveau type de shaders en OpenGL 3.x (mais pas en OpenGL ES 3.x) : le Geometry Shader.
Il s'insère après le Vertex Processing mais avant la phase de Génération de Primitive et le Rasterizer.
Il reçoit en entrée chaque primitive sous forme d'un tableau d'attributs (3 sommets si GL_TRIANGLES, 2 sommets si GL_LINES, 1 sommet si GL_POINTS).
En sortie il peut émettre, en théorie, un nombre illimité de points (faire de la génération) mais en pratique les constructeurs limitent en mémoire et donc en nombre le nombre d'attributs que l'on peut produire.
Assez rarement utilisé de nos jours, essentiellement parce que délaissé par les constructeurs -ce qui lui confère relativement de faibles performances- il n'en reste pas moins utile pour différentes tâches que ce soit du debug de primitive ou de la visualisation d'arêtes, ou faire de l'extrusion etc…

Le fait que le Geometry Shader s'insère après le Vertex Shader et avant la rasterization a fait que certains mots clés du GLSL sont devenus obsolètes ou incohérents.
Par exemple varying est censé indiquer que les données en sortie du Vertex Shader vont « varier » du fait du Rasterizer.
Or maintenant la sortie du Vertex Shader peut être l'entrée du Geometry Shader. Si varying a dès lors du sens dans le Geometry Shader, ce n'est plus le cas dans le Vertex Shader puisqu'un Geometry Shader récupère les sorties du Vertex Shader « tel quel ».

La syntaxe du GLSL à donc évoluer et maintenant l'ensemble des entrées ('attribute' du Vertex Shader, ou 'varying' du Fragment Shader) utilise maintenant le mot clé '**in'**, tandis que l'ensemble des sorties ('varying' du Vertex Shader ou Geometry Shader, et gl_FragColor dans le Fragment Shader) utilise désormais le mot clé '**out'**.
Il faut au minimum indiquer le numéro de version 140 (OpenGL 3.1) mais je vous recommande de spécifier #version 330 qui correspond à OpenGL 3.3, la version la plus stable et répandue d'OpenGL 3.

Nouveautés du pipeline d'OpenGL 4.x et au-delà